Software para Sistemas Ubíquos · Instituto de Informática, UFG · Prof. Dr. Otávio Calaça Xavier

Permanência medida, não presença num instante

O Marco 2 do sistema Presença: um ESP32 no papel do celular do estudante, que ouve as três âncoras da sala, e um servidor que transforma esses heartbeats em minutos de permanência, avisa antes de punir e devolve a decisão ao aparelho. Cada arquivo, cada peça, cada regra e cada teste executado no Wokwi, com as capturas e os logs reais.

Aluna **Alany Gabrielly** · 202105018 Responsabilidade **comunicação e resiliência**

Teste adversarial **sequência** , matrícula terminada em 8 Executado em **25/09/2026**

Parte I · fronteira principal

Aparelho do estudante ↔ servidor de permanência

O núcleo do sistema Presença rodando de verdade: um ESP32 que ouve as três âncoras e um servidor que transforma esses heartbeats em minutos de permanência, avisa antes de punir e devolve a decisão ao aparelho.

01 O que este marco entrega

A chamada tradicional mede **presença**: um estado binário, num instante arbitrário. O Presença mede **permanência**, o tempo acumulado ao longo da aula. Três minutos de sensor ruim custam três minutos, e não a aula inteira.

O Marco 2 pede uma fronteira real entre *dois* componentes. A escolhida é a que carrega o projeto: o **aparelho do estudante**, que mede o sinal das âncoras, falando com o **servidor**, que decide quanto tempo aquilo vale. Tudo o que o projeto técnico chama de regra inviolável passa por essa fronteira: histerese,

trilateração, polígono, persistência, acumulador monotônico, orçamento por sessão, requalificação e aviso antes da punição.

7	34	98,6 %	3
cenários executados no Wokwi, com captura e log	testes automatizados passando	permanência da sessão gravada, calculada sobre 188 heartbeats	defeitos encontrados rodando, todos corrigidos

O ciclo completo exigido pela Atividade 03

ETAPA	ONDE ACONTECE
Fenômeno	Intensidade do sinal das âncoras A, B e C no ponto onde o estudante está
Entrada	Três potenciômetros em `GPIO34`, `GPIO35` e `GPIO32`, convertidos para −105… −35 dBm
Validação	Âncora não ouvida abaixo de −100 dBm (dado ausente, não número baixo), média móvel de 5 amostras por âncora
Evento	`presenca.heartbeat.v1` em JSON, no serial e no MQTT, a cada 500 ms
Estado e regra	No aparelho, região com histerese −70/−85 dBm. No servidor, polígono, persistência, máquina de estados, orçamento e requalificação
Decisão	PRESENTE · AUSENCIA_TOLERADA · FORA
Atuação	LEDs e OLED mostram o estado; buzzer e OLED entregam os avisos de 10 min, 13 min e limite esgotado

02 A fronteira

Produtor

Aparelho do estudante

ESP32 · C++ · roda no Wokwi

- Mede o RSSI de cada âncora
- Trata âncora não ouvida
- Filtra e aplica histerese de região
- Envia heartbeat com sequência

▶ heartbeat a cada 500 ms

`presenca.heartbeat.v`

MQTT · broker.hivemq.com

◀ estado a cada heartbeat

Consumidor

Servidor de permanência

Python · roda no notebook

- Valida contra o contrato
- Deduplica, ordena, acha lacunas
- Trilatera e testa o polígono
- Refaz a sessão sobre o log

- Retém heartbeats se a rede cai
- Mostra o estado, toca o aviso

`presenca.estado.v1`

◀ só em 4 momentos

`presenca.aviso.v1`

- Orçamento e requalificação
- Avisa e detecta silêncio

A âncora da sala **não** aparece nesta fronteira, e isso é deliberado: pela decisão fechada nº 3, ela só transmite e nunca escuta aparelhos. Quem mede é o aparelho. Por isso o ESP32 deste marco faz o papel do celular, e as âncoras existem apenas como o sinal que ele ouve (os potenciômetros).

Quem decide o quê

DECISÃO	ONDE	POR QUÊ
A âncora foi ouvida? Qual o sinal filtrado?	Aparelho	É medição; só o aparelho tem o rádio
Região de rádio: dentro ou fora do alcance	Aparelho	É o que o iOS faz no monitoramento de região. O sistema operacional decide entrada e saída e acorda o app
O ponto está *dentro da sala*?	Servidor	Só o servidor conhece a planta. Rádio atravessa parede, e um sinal forte na sala vizinha passa pela histerese mas não pelo polígono
Estado, acumulado, orçamento, avisos	Servidor	Precisa do log inteiro da sessão e é recalculado retroativamente
Qual horário vale	Servidor	O aparelho só informa tempo relativo ao boot; o servidor grava o próprio carimbo de recepção

Uma volta completa pela fronteira

1 Aparelho · a cada 100 ms

Lê os três ADCs, converte para dBm, descarta âncora abaixo de −100 dBm e atualiza a média móvel de cada uma.

2 Aparelho

Aplica a histerese sobre o melhor sinal: `DENTRO_ALCANCE` acima de −70 dBm, `FORA_ALCANCE` abaixo de −85 dBm. Entre os dois, mantém.

3 **Aparelho · a cada 500 ms**

Monta o heartbeat com `eventId = apar-01#bootId#seq` e publica. Se a rede está fora, guarda na fila.

4 **Servidor**

Valida contra `presenca.heartbeat.v1.schema.json`. Um campo a mais ou a menos é rejeitado.

5 **Servidor**

Confere a sequência: duplicata é descartada, fora de ordem é inserido no lugar certo, salto abre uma lacuna.

6 **Servidor**

Converte RSSI em distância, trilatera, testa o polígono e classifica o heartbeat como dentro, fora ou neutro.

7 **Servidor**

Refaz a sessão inteira a partir do log e obtém estado, acumulado garantido, provisório e orçamento restante.

8 **Servidor → aparelho**

Publica `presenca.estado.v1` com retain, no tópico do próprio aluno. Se um limite de aviso foi cruzado, publica também `presenca.aviso.v1`.

9 **Aparelho**

Acende o LED do estado, escreve no OLED e, se houver aviso novo, toca o buzzer. Aviso repetido é ignorado.

03 Todos os arquivos

```
marco-02/
├── README.md                    ← a entrega: a fronteira em uma página
├── GUIA.md                      ← guia da segunda fronteira (CO₂)
│
├── contrato/v1/                 ← O IDIOMA dos componentes (versão no nome)
│   ├── presenca.heartbeat.v1.schema.json   aparelho → servidor
│   ├── presenca.estado.v1.schema.json      servidor → aparelho
│   ├── presenca.aviso.v1.schema.json       servidor → aparelho
│   ├── sala.ambiente.v1.schema.json        (CO₂) âncora → gateway
│   ├── gateway.contagem.v1.schema.json     (CO₂) gateway → âncora
│   └── exemplos/                mensagens reais, todas válidas pelo schema
│
├── aparelho/                    ← O QUE RODA NO ESP32 (Wokwi)
│   ├── sketch.ino                   ~535 linhas de C++
│   ├── diagram.json                 o circuito em texto
│   └── libraries.txt                PubSubClient, Adafruit GFX, SSD1306
│
├── servidor/                    ← O QUE RODA NO NOTEBOOK
│   ├── permanencia.py               a regra: localização + máquina de estados, se
│   ├── servidor.py                  MQTT, contrato, sequência, avisos, silêncio
│   ├── config.exemplo.json          broker, tópicos, parâmetros, planta da sala
│   ├── requirements.txt             paho-mqtt, jsonschema
│   └── testes/                      34 testes (pytest)
│
├── co2-ancora/  co2-gateway/    ← a segunda fronteira (Parte II)
│
├── evidencias/presenca/         ← capturas e logs da execução no Wokwi
│   ├── 01…07-*.jpg                  uma captura por cenário
│   ├── log-serial-aparelho.txt      tudo o que o ESP32 escreveu
│   ├── log-servidor.txt             tudo o que o servidor escreveu
│   └── recebidos.jsonl              cada heartbeat com carimbo e classificação
│
└── apresentacao/                ← este documento (HTML e PDF) e suas fontes
```

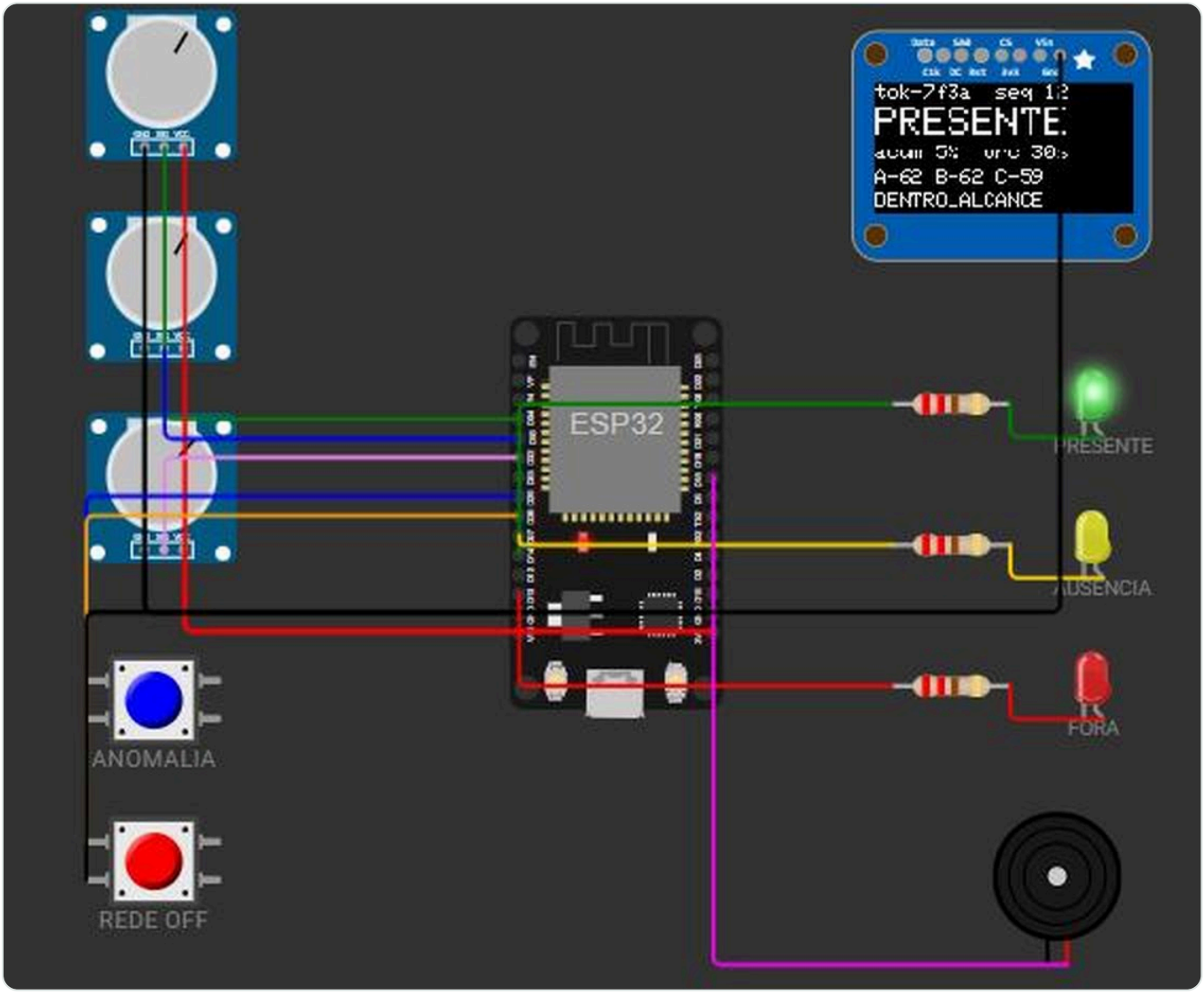
Por que a lógica mora num arquivo sem rede

`permanencia.py` não importa MQTT nem lê relógio. Recebe uma lista de heartbeats e devolve o resultado. Isso tem duas consequências. Dá para testar cada regra do projeto técnico em milissegundos, sem Wokwi nem broker. E o cálculo retroativo sai de graça: como a função só olha para o log, basta chamá-la de novo quando chega um dado atrasado.

Arquivos que nascem em execução

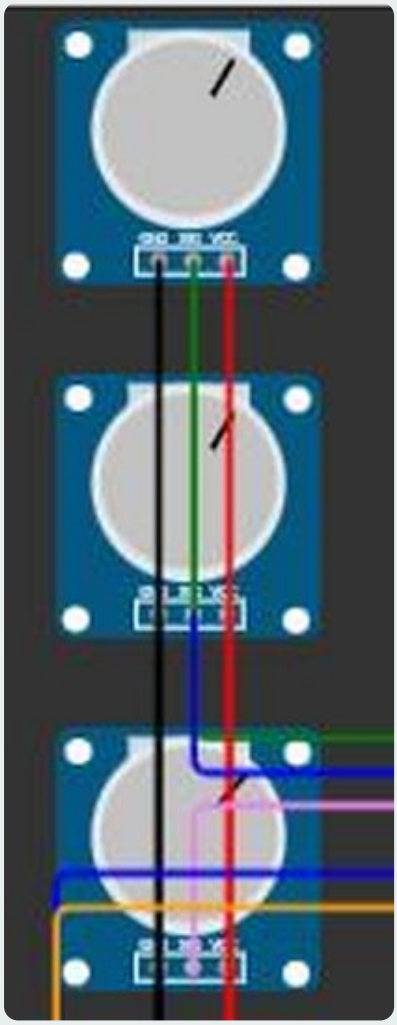
ARQUIVO	QUANDO APARECE	VERSIONADO?
`servidor/config.json`	Ao copiar o exemplo para mudar broker ou escala	Não
`servidor/recebidos.log`	Toda execução; uma linha JSON por heartbeat, com o carimbo do servidor, a classificação e o que foi derivado (t, classe, posição, estado)	Não, mas uma cópia está em `evidencias/`

04 O circuito do aparelho, peça por peça



Circuito completo em execução. O OLED mostra `PRESENTE` e o LED verde está aceso: o servidor confirmou a presença.

PEÇA	PINO DO ESP32	PAPEL
Potenciômetro A (de cima)	`GPIO34`	RSSI da âncora `anc-204-A`, parede oeste
Potenciômetro B (meio)	`GPIO35`	RSSI da âncora `anc-204-B`, parede leste
Potenciômetro C (de baixo)	`GPIO32`	RSSI da âncora `anc-204-C`, parede norte
Botão azul ANOMALIA	`GPIO25`	Arma duplicata → fora de ordem → salto
Botão vermelho REDE OFF	`GPIO26`	Liga e desliga a rede simulada
LED verde + 220 Ω	`GPIO14`	`PRESENTE`
LED amarelo + 220 Ω	`GPIO27`	`AUSENCIA_TOLERADA`; pisca durante um aviso
LED vermelho + 220 Ω	`GPIO13`	`FORA`
Buzzer	`GPIO18`	Toca 400 ms a 1760 Hz em cada aviso novo
OLED SSD1306 (I²C 0x3C)	`SDA 21` `SCL 22`	Estado, acumulado, orçamento, RSSI, região e texto do aviso



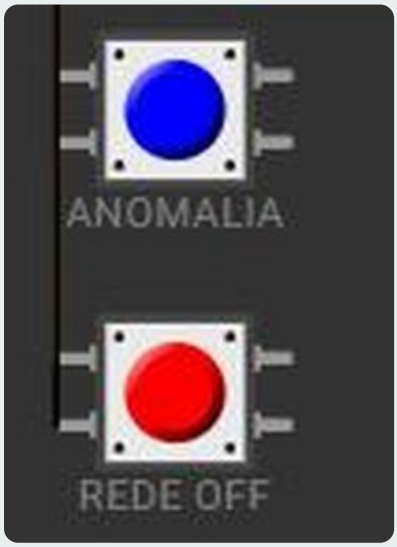
Três potenciômetros: o sinal das três âncoras

O que são. Resistores variáveis com três pinos: VCC (3,3 V), GND e o cursor (SIG), que entrega uma tensão entre 0 e 3,3 V conforme o giro. O ADC de 12 bits do ESP32 lê isso como 0 a 4095.

O que representam. Cada um é o RSSI que o celular mediria de uma âncora. O código converte linearmente: `-105 + (leitura / 4095) × 70`, ou seja, −105 dBm no mínimo e −35 dBm no máximo.

Por que três. Porque o projeto usa trilateração: com três distâncias dá para calcular um ponto, e com um ponto dá para perguntar se ele está *dentro da sala*. Um potenciômetro só daria um raio, e raio é exatamente o que o projeto rejeita.

Por que ADC1. Os pinos 32, 34 e 35 pertencem ao ADC1. O ADC2 fica indisponível enquanto o Wi-Fi está ligado, e este aparelho vive conectado.

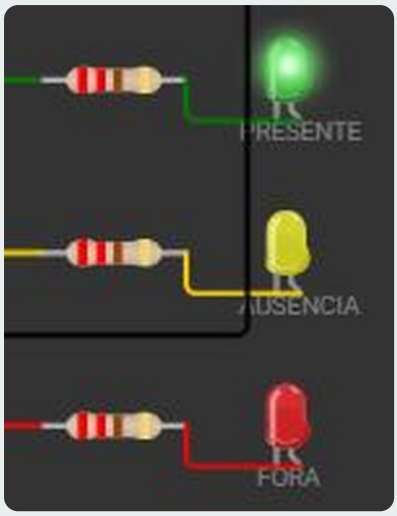


Dois botões: as falhas sob demanda

Ligados entre o pino e o GND, com o `INPUT_PULLUP` interno: solto lê `HIGH`, apertado lê `LOW`. O código age na **borda de descida** e ignora mudanças a menos de 50 ms da anterior (debounce), para que um aperto não conte como vários.

Azul, ANOMALIA. Cada aperto arma a próxima anomalia do ciclo duplicata → fora de ordem → salto, aplicada no heartbeat seguinte. É o teste adversarial da matrícula terminada em 8.

Vermelho, REDE OFF. Alterna uma queda de rede simulada. O aparelho continua medindo e guarda os heartbeats; ao voltar, entrega todos com `bufferizado: true`.



Três LEDs: a decisão do servidor, visível

Cada LED tem um resistor de 220 Ω em série: com 3,3 V no pino e cerca de 2 V de queda no LED, passam uns 6 mA, dentro do que o GPIO fornece com folga.

Comportamento seguro. Os LEDs mostram o estado que o servidor decidiu, e só enquanto essa resposta é recente (4 s). Se o servidor some ou a rede cai, **todos se apagam**: um verde velho aceso diria "você está presente" sem ninguém saber se é verdade. O OLED explica o motivo (`SEM RESP.` ou `REDE OFF`).



Buzzer: o aviso que fecha o laço

Um piezo entre o `GPIO18` e o GND. `tone(18, 1760, 400)` gera uma onda quadrada de 1760 Hz por 400 ms.

Ele representa a notificação do celular. É o ponto em que o sistema deixa de só medir e passa a **atuar**: o aviso chega enquanto o estudante ainda pode voltar, e voltar muda a grandeza que o sistema mede. Por isso ele toca só nos momentos previstos pelo projeto, e uma vez só por aviso.

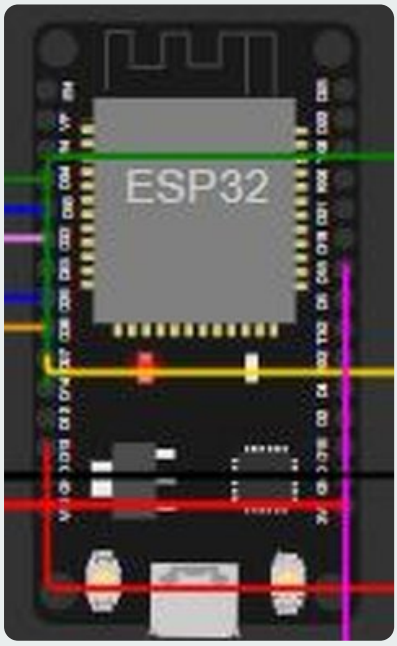


OLED: o que o estudante vê

SSD1306 de 128×64 pixels, pelo barramento I^2C (SDA no 21, SCL no 22, endereço 0x3C).

LINHA	CONTEÚDO
1	Pseudônimo do aluno e sequência atual
2 (grande)	Estado decidido pelo servidor, `REDE OFF` ou `SEM RESP.`
3	Acumulado (%) e orçamento restante (s), ou a fila retida
4	RSSI filtrado de A, B e C (`--` = não ouvida)
5	Região decidida no próprio aparelho

Durante um aviso, as linhas 3 a 5 dão lugar ao texto da mensagem por 6 s.



ESP32: o aparelho

Processador de dois núcleos com Wi-Fi. No Wokwi, conecta à rede `Wokwi-GUEST`, que tem saída real para a internet. É por isso que o MQTT funciona com um broker público de verdade.

Ele não guarda identidade: o aluno aparece só como o pseudônimo `tok-7f3a`. No sistema real, esse identificador vem da chave pública gerada no cofre de hardware no pareamento. Aqui é fixo, e não há assinatura.

05 Como o programa do aparelho funciona

Leitura e validação: âncora não ouvida é falta de dado

```
if (bruto < RSSI_NAO_OUVIDA) {         // abaixo de -100 dBm
  ouvida[i] = false;
  preenchida[i] = 0; indice[i] = 0;     // esvazia a janela desta âncora
  continue;
}
```

Uma implementação ingênua trataria −103 dBm como "muito longe" e jogaria o número na média. Mas −103 dBm, aqui, significa que o aparelho **não ouviu** a âncora. Não é uma medida baixa, é ausência de medida. O heartbeat leva `"rssi": null` e a janela é esvaziada, para que valores antigos não se misturem aos novos quando a âncora voltar. Leituras fisicamente impossíveis (acima de −30 dBm) contam em `amostrasInvalidas`.

Média móvel por âncora

Cinco amostras de 100 ms. Suaviza o ruído sem atrasar mais que meio segundo. A região só é avaliada com a janela cheia; antes disso o estado é `INDETERMINADA`, e o servidor não decide nada com base nele.

Regra local: histerese de região

```
if (nenhumaOuvida)                  nova = REG_FORA;
else if (melhor > LIMIAR_ENTRADA)   nova = REG_DENTRO;    // -70 dBm
else if (melhor < LIMIAR_SAIDA)     nova = REG_FORA;      // -85 dBm
// entre os dois: mantém
```

Dois limiares, não um. Quem senta na borda do alcance, com o sinal oscilando entre −72 e −78 dBm, não fica entrando e saindo: a faixa morta de 15 dB absorve a oscilação. Esta é a característica de estado que a Atividade 03 exige no próprio ESP32 (item 5.3).

O heartbeat e a sequência

`eventId = deviceId#bootId#sequence`. O `bootId` é sorteado com `esp_random()` a cada inicialização. Sem ele, reiniciar o aparelho zeraria a

sequência, e todo heartbeat novo teria a mesma identidade de um antigo: o servidor descartaria o boot inteiro como cópia. O problema foi visto rodando num protótipo anterior, e a correção vale para os dois.

A fila da queda de rede

Com a rede fora, o heartbeat vai para uma fila circular de 60 posições (30 s), guardando a sequência, o `eventTimeMs` original, os três RSSI e a região. Se a fila enche, o **mais antigo** é descartado e contado. O servidor verá um salto na sequência, e o intervalo aparece como sem evidência, e não como vazio. Ao voltar, a fila é drenada com `bufferizado: true`.

As três anomalias, feitas de verdade

ANOMALIA	COMO O APARELHO A PRODUZ
Duplicata	Reenvia o *último payload*, byte a byte: mesmo `eventId`, mesmo conteúdo. É o que um QoS 1 reentregando faria
Fora de ordem	Segura o heartbeat N e o publica logo depois do N+1. A mensagem é real e nova, só chega na ordem errada
Salto	Soma 5 à sequência. Esses cinco números nunca existem

Recebendo as respostas

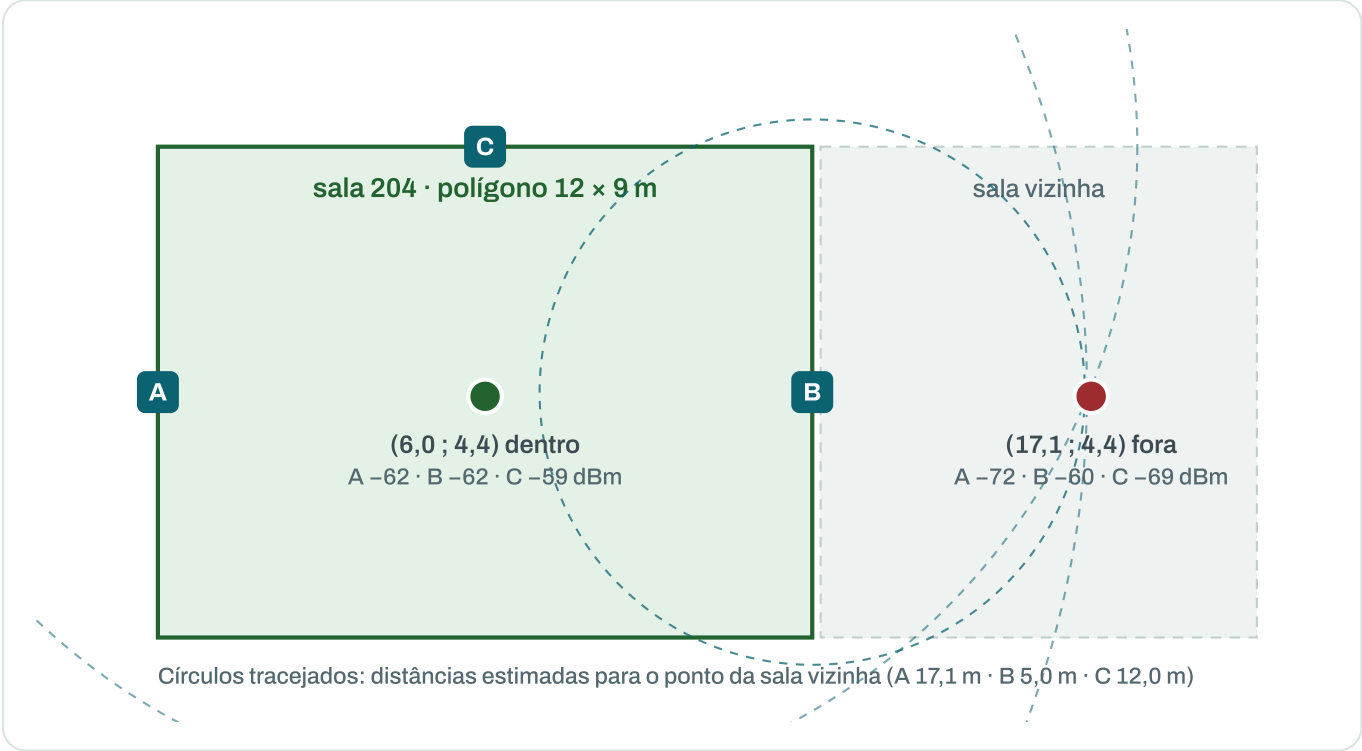
O parser procura `"campo":valor` na string recebida. É pequeno e não aloca árvore JSON, o que importa num microcontrolador. Duas proteções:

- **Estado velho não sobrescreve estado novo.** `estadoId` traz um número crescente; um número menor que o último aplicado é ignorado (a não ser que a queda seja grande, o que indica servidor reiniciado).
- **Aviso é idempotente.** Os últimos 8 `avisoId` exibidos ficam guardados; uma reentrega não toca o buzzer de novo.

06 Como o servidor decide

Onde o aparelho está: trilateração e polígono

Cada RSSI vira uma distância pelo modelo de propagação do projeto técnico (§5): `d = 10^((A − RSSI) / 10n)`, com `A = −45 dBm` a 1 m e `n = 2,2`. Três círculos, subtraídos dois a dois, viram um sistema linear em (x, y), resolvido direto. O ponto então passa por um teste de pertinência ao polígono (*ray casting*): conta quantas paredes uma semirreta a partir do ponto atravessa.

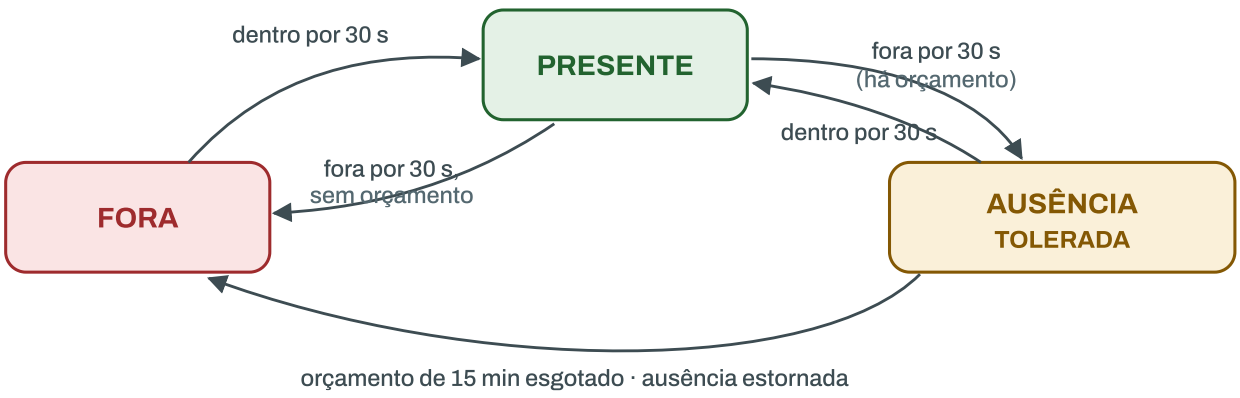


O ponto vermelho é o caso que derruba qualquer limiar em metros. O melhor sinal é de −60 dBm, forte o bastante para passar pela histerese do aparelho (`DENTRO_ALCANCE`). Um raio de alcance contaria presença. Mas os três círculos se cruzam a 5 m *do lado de fora* da parede leste, e o polígono diz não. Esse cenário foi executado no Wokwi (teste T2).

O que cada heartbeat diz

REGIÃO INFORMADA	POSIÇÃO	CLASSE
`FORA_ALCANCE`	qualquer, ou nenhuma	**fora**
`DENTRO_ALCANCE`	dentro do polígono	**dentro**
`DENTRO_ALCANCE`	fora do polígono	**fora**
`DENTRO_ALCANCE`	sem as três âncoras	**neutro**: não há base para decidir
`INDETERMINADA`	—	**neutro**

A máquina de estados



PAUSA_PROGRAMADA existe no contrato, mas depende da agenda e não está neste marco

O acumulado é refeito sobre o log, a cada heartbeat

O servidor nunca faz `acumulado += 0,5`. A cada heartbeat aceito, ele ordena o log inteiro pelo tempo do evento e percorre a sessão desde o início. Isso é o que torna possível cumprir três regras do projeto técnico ao mesmo tempo:

- **Entrada e saída valem desde o início real.** A saída só é confirmada depois de 30 s de sinal fora, mas ela começou 30 s antes. Esses 30 s deixam de ser presença e passam a ser ausência, retroativamente.
- **Evento atrasado ou fora de ordem não corrompe nada.** Ele entra na posição certa do log, e o resultado é idêntico ao da chegada em ordem. Há um teste que embaralha o log inteiro e compara.

- **O acumulado garantido nunca diminui.** O que ainda pode mudar (ausência em curso, ausência aguardando requalificação, saída ainda não confirmada) é publicado à parte, como `provisorioS`.

Orçamento, requalificação e estorno

SITUAÇÃO	O QUE ACONTECE COM O TEMPO
Sai e volta com orçamento	A ausência consome o orçamento e fica **provisória**. Só é creditada se o estudante permanecer `min(30 min, restante)` depois de voltar
Volta só por instantes e sai de novo	A ausência anterior é **estornada**, mas o orçamento continua consumido. Aparecer 20 s não reabre crédito
Orçamento acaba durante a ausência	Vai para `FORA` e a ausência **inteira** é estornada. O aviso informa quando o tempo parou de contar: no início real da ausência
Várias saídas curtas	O orçamento é da **sessão**, não do evento. Três saídas de 6 min estouram os 15
Sai no fim da aula	Soma o que assistiu. Ausência em curso no encerramento não é creditada

Os quatro avisos

Só existem os quatro momentos do §7 do projeto técnico: 10 min de ausência acumulada, 13 min, limite esgotado e fim da aula. Cada um é emitido **uma vez por sessão**, com `avisoId = sessão#tipo`. Se o estudante ignorar, o sistema não insiste.

Silêncio

Uma thread do servidor observa o relógio real. Se nenhum heartbeat chega por 6 s, o servidor escreve `SILENCIO` e **não decide nada**: silêncio pode ser rede, bateria ou aplicativo morto, e nenhuma dessas é evidência de que o estudante saiu. O que chegar depois entra no passado. Quando os heartbeats voltam, escreve `RETOMADO`.

O tempo do protótipo

O relógio do Wokwi anda entre 10 % e 80 % do tempo real, variando com a carga. Se o servidor medisse durações pelo próprio relógio, "30 s de ausência"

seria outra coisa a cada execução. Por isso a linha do tempo da máquina de estados é o `eventTimeMs` do aparelho, usado **só como diferença** dentro de um mesmo boot. Um boot novo é encaixado logo depois do último instante conhecido. O carimbo do servidor continua gravado em cada linha do log, para auditoria. Num aparelho real, com relógio sincronizado, a máquina usaria o carimbo do servidor.

07 Os contratos

Três schemas JSON (draft 2020-12) em `contrato/v1/`, todos com `additionalProperties: false`. O servidor lê o schema do disco e valida cada heartbeat: um campo inventado é rejeitado, não ignorado. Os exemplos em `contrato/v1/exemplos/` são mensagens **reais** da sessão gravada, e todos passam pela validação.

`presenca.heartbeat.v1` : aparelho → servidor

```
{
  "eventType": "presenca.heartbeat.v1",
  "eventId": "apar-01#a52e#14",           ← deviceId#bootId#sequence
  "deviceId": "apar-01",
  "entityId": "sala-204",
  "aluno": "tok-7f3a",                    ← pseudônimo, nunca MAC/IMEI/nome
  "bootId": "a52e",
  "eventTimeMs": 16371,                   ← desde o boot, não absoluto
  "sequence": 14,
  "value": -60.4,  "unit": "dBm",         ← melhor sinal
  "state": "DENTRO_ALCANCE",              ← histerese no aparelho
  "ancoras": [
    {"id": "anc-204-A", "rssi": -72.1},
    {"id": "anc-204-B", "rssi": -60.4},
    {"id": "anc-204-C", "rssi": -68.7}
  ],
  "amostrasInvalidas": 0,
  "bufferizado": false,
  "anomaliaInjetada": "nenhuma"           ← o servidor não lê este campo
}
```

Este é o heartbeat da sala vizinha: dentro do alcance e, mesmo assim, classificado como **fora**.

`presenca.estado.v1` : servidor → aparelho, com retain

```
{"eventType":"presenca.estado.v1","estadoId":"srv-204#62","aluno":"tok-7f3a",
 "entityId":"sala-204","estado":"AUSENCIA_TOLERADA","acumuladoS":7.4,"provisorioS"
 "percentual":7.4,"orcamentoRestanteS":5.5,"tempoAulaS":33.3,"classe":"fora",
 "posicao":[17.1,4.4],"dentroPoligono":false,"encerrada":false,"emitidoEmMs":17903
```

Publicado no tópico `…/estado/tok-7f3a` : cada aparelho só assina o próprio. Com `retain` , um aparelho que acabou de ligar recebe o último estado de imediato. O JSON sai compacto, sem espaços, porque o parser do ESP32 procura `"campo":valor` .

`presenca.aviso.v1` : servidor → aparelho, QoS 1

```
{"eventType":"presenca.aviso.v1","avisoId":"s31a2#AVISO_10_MIN","aluno":"tok-7f3a"
 "tipo":"AVISO_10_MIN","texto":"Fora da sala ha 20 s. Seu limite e de 30 s nesta a
 "percentual":7.4,"emitidoEmMs":1790310788000}
```

Por que a versão está no nome

Uma mudança incompatível cria `presenca.heartbeat.v2` , e a v1 continua válida enquanto os aparelhos antigos migram. Adicionar `bateria` ao heartbeat hoje faz o servidor responder `INVALIDO … Additional properties are not allowed` : há um teste só para isso. É a alteração ao vivo mais convincente para mostrar na banca.

08 Testes executados no Wokwi

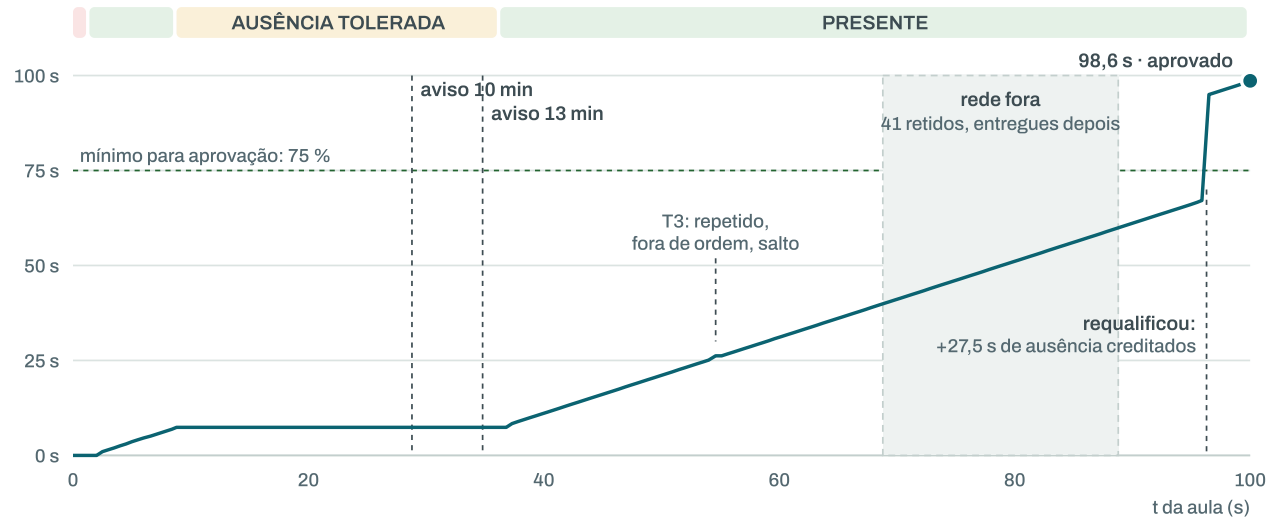
Tudo abaixo foi executado em 25/09/2026, com o aparelho no Wokwi e o servidor no notebook falando pelo `broker.hivemq.com:1883` . Não há simulação de resultado: as capturas são da tela do Wokwi, e os trechos de terminal foram copiados dos logs em `evidencias/presenca/` . Os potenciômetros e botões foram acionados pelo próprio circuito do Wokwi.

Parâmetros de todos os testes

Escala 1:30. Persistência 1 s · orçamento 30 s · avisos em 20 s e 26 s · requalificação 60 s · aula de 100 s · mínimo de 75 %. Heartbeat a cada 500 ms. Tempos em segundos da linha do tempo da sessão (`t`).

A sessão gravada, do início ao fim

Os testes T1 a T4 e T6 aconteceram numa mesma aula de 100 s, a sessão `s31a2`. O gráfico mostra o acumulado garantido que o servidor publicou para cada heartbeat, na ordem do tempo do evento.

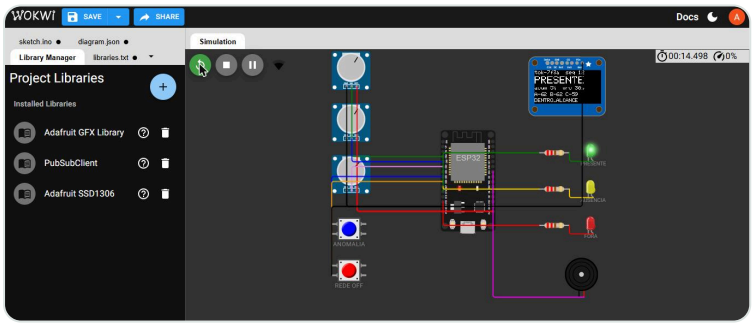


Três coisas para notar. Durante a ausência (faixa âmbar) o acumulado **para**, mas não cai. Na queda de rede, o servidor ficou em silêncio e depois recebeu 41 heartbeats atrasados que preencheram o trecho sem buraco. Em t = 96,3 s a requalificação se cumpre e os 27,5 s de ausência tolerada entram de uma vez no acumulado.

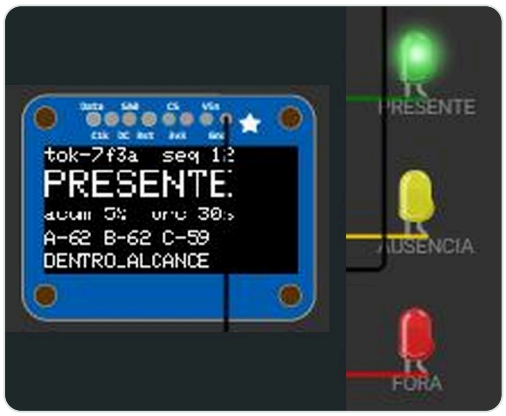
T1 Estado normal: dentro da sala

passou

Estado inicial	Simulação reiniciada, sessão nova `s31a2` aberta no servidor. Estado FORA, acumulado 0.
Entradas	Potenciômetros em A = 628, B = 628, C = 667 (de 1023), o centro da sala.
Valores e tempos	RSSI filtrado A −62,0, B −62,0, C −59,4 dBm. Posição trilaterada (6,0; 4,4). Região `DENTRO_ALCANCE`.
Esperado	Depois de 1 s de sinal dentro, `PRESENTE` desde o início real da condição; LED verde; acumulado crescendo; nenhum aviso.
Observado	`FORA → PRESENTE em t=1,4 s`, confirmado no heartbeat seguinte e datado no início da condição. LED verde aceso, OLED com `PRESENTE`, orçamento intacto em 30 s.
Por quê	Três heartbeats seguidos *dentro* cobrem a persistência. A entrada é datada no primeiro deles, e não no que a confirmou: é o cálculo retroativo.



Captura do Wokwi durante o T1.



OLED e LEDs ampliados.

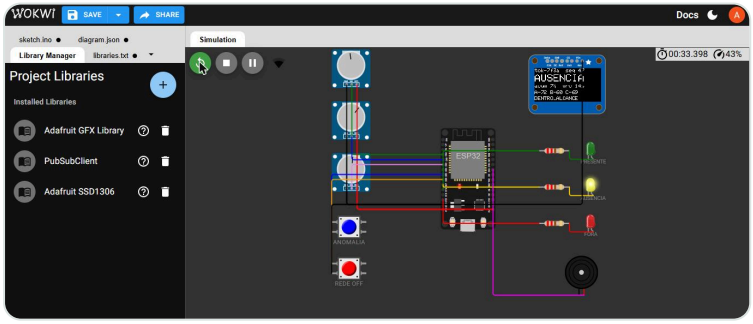
servidor · log-servidor.txt

```
01:30:53 t=  2.0s seq=3   A -62 B -62 C -59 DENTRO_ALCANCE ( 6.0, 4.4) dentro -> FORA
01:30:55 t=  2.5s seq=4   A -62 B -62 C -59 DENTRO_ALCANCE ( 6.0, 4.4) dentro -> PRESENTE  acum  1.
   >>> FORA -> PRESENTE em t=1.4 s (sinal na sala sustentado)
```

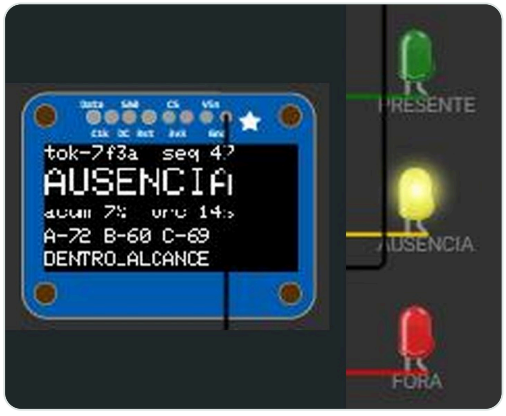
T2 Decisão e atuação: sala vizinha, avisos e retorno

passou

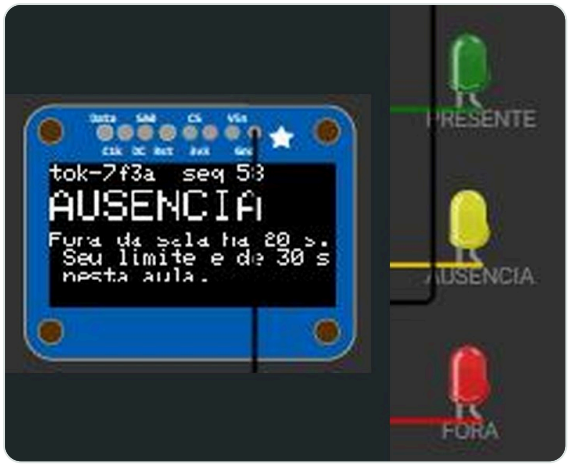
Estado inicial	`PRESENTE`, acumulado 7,4 s, orçamento 30 s.
Entradas	Potenciômetros para A = 481, B = 652, C = 531 (sala vizinha, colado à parede leste). Depois do segundo aviso, de volta a 628 / 628 / 667.
Valores e tempos	RSSI A −72, B −60, C −69 dBm: a região continua `DENTRO_ALCANCE`. Posição (17,1; 4,4), fora do polígono. Saída a partir de t = 8,8 s; volta em t = 36,3 s.
Esperado	`AUSENCIA_TOLERADA` com LED amarelo; aviso de 10 min em 20 s de ausência (t = 28,8); aviso de 13 min em 26 s (t = 34,8); retorno antes dos 30 s volta a `PRESENTE` sem punição; ausência provisória até a requalificação.
Observado	Exatamente isso. Buzzer e OLED com "Fora da sala ha 20 s. Seu limite e de 30 s nesta aula." e depois "Faltam 4 s do seu limite...". Retorno confirmado em t = 36,3 s com 2,5 s de orçamento sobrando. A ausência de 27,5 s foi creditada em t = 96,3 s, quando a requalificação se cumpriu.
Por quê	A histerese sozinha diria "presente": o sinal é forte. É o polígono que decide. Os avisos chegam enquanto ainda cabe reação, e a reação (voltar) é que impede a punição. É o laço ciber-físico fechado.



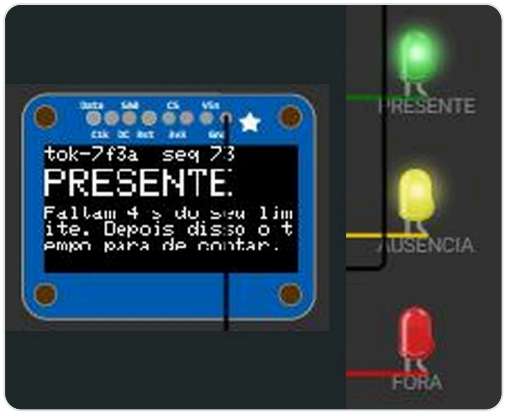
Sinal forte, mas na sala vizinha: **AUSENCIA**, com a região ainda `DENTRO_ALCANCE`.



OLED e LEDs ampliados.



Primeiro aviso (10 min reais, 20 s na escala).



Segundo aviso, já de volta: o LED verde acendeu.

servidor · log-servidor.txt (trechos)

```
01:32:36 t= 17.0s seq=30  A -72 B -60 C -69 DENTRO_ALCANCE ( 17.1,  4.4) fora  -> AUSENCIA_TOLERAD
 AVISO AO ESTUDANTE [AVISO_10_MIN] t=28.8 s
 "Fora da sala ha 20 s. Seu limite e de 30 s nesta aula."
 AVISO AO ESTUDANTE [AVISO_13_MIN] t=34.8 s
 "Faltam 4 s do seu limite. Depois disso o tempo para de contar."
01:33:26 t= 36.3s seq=68  A -62 B -62 C -59 DENTRO_ALCANCE (  6.0,  4.4) dentro -> AUSENCIA_TOLERA
01:33:27 t= 36.8s seq=69  ...                                  dentro -> AUSENCIA_TOLERADA acum  7.4s orc 2.0
01:33:28 t= 37.3s seq=70  ...                                  dentro -> PRESENTE       acum  8.4s orc 2.5s
   >>> AUSENCIA_TOLERADA -> PRESENTE em t=36.3 s (retornou dentro do orcamento)
01:36:37 t= 95.9s seq=182 ...                                   dentro -> PRESENTE       acum 67.1s
01:36:38 t= 96.5s seq=183 ...                                   dentro -> PRESENTE       acum 95.0s  ← requalific
```

Repare no orçamento em t = 36,8 s: 2,0 s. No heartbeat seguinte, quando o retorno se confirma, ele **volta a 2,5 s**. O servidor refez a sessão e descobriu que a ausência terminou em 36,3 s, e não em 36,8 s.

T3 Adversarial (matrícula final 8): repetido, fora de ordem, salto

passou

Estado inicial	`PRESENTE`, t ≈ 49 s, acumulado 20,5 s, sequência 91.
Entradas	Três apertos do botão azul, cada um segurado por 2 s, com cerca de 6 s entre eles.
Valores	Duplicata do seq 92 · seq 100 retido e entregue depois do 101 · salto de 106 para 112.
Esperado	Duplicata descartada sem recálculo; fora de ordem aceito na posição original, sem alterar o resultado; salto contado com 5 perdidos. Nenhuma mudança de estado.
Observado	`DUPLICADO` · `FORA DE ORDEM … inserido na posicao original` · `SALTO … 5 heartbeat(s) perdido(s)`. Estado `PRESENTE` o tempo todo. O fora de ordem chegou com o mesmo acumulado do heartbeat anterior (26,2 s), já que o resultado não depende da ordem de chegada.
Por que o ingênuo erra	Um contador que soma 0,5 s por heartbeat recebido contaria a duplicata duas vezes. Um que usa "o último que chegou" como estado atual retrocederia no tempo com o fora de ordem. Um que não olha a sequência não saberia que 5 heartbeats sumiram, e trataria o intervalo como observado.

aparelho · log-serial-aparelho.txt

```
### anomalia armada: duplicata (aplica no proximo heartbeat)
>>> ADVERSARIAL: reenviando o ultimo heartbeat (mesmo eventId)
### anomalia armada: fora-de-ordem (aplica no proximo heartbeat)
>>> ADVERSARIAL: segurando seq=100 para entregar depois de 101
>>> ADVERSARIAL: entregando agora o heartbeat retido (fora de ordem)
### anomalia armada: salto (aplica no proximo heartbeat)
>>> ADVERSARIAL: salto de 106 para 112 (5 numeros nunca enviados)
```

servidor · log-servidor.txt

```
01:35:06 t= 49.9s  seq=92   ... dentro -> PRESENTE  acum  21.0s
01:35:07 DUPLICADO     apar-01#a52e#92 ja processado - descartado, nada recalculado
01:35:17 SALTO         esperado 100, recebido 101 - 1 heartbeat(s) perdido(s); o intervalo fica sem evidencia
01:35:17 t= 55.1s  seq=101  ... dentro -> PRESENTE  acum  26.2s
01:35:17 FORA DE ORDEM  seq=100 chegou depois de 101 - inserido na posicao original e sessao recalcula
01:35:17 t= 54.6s  seq=100  ... dentro -> PRESENTE  acum  26.2s
01:35:24 SALTO         esperado 107, recebido 112 - 5 heartbeat(s) perdido(s); o intervalo fica sem evidencia
01:35:24 t= 58.1s  seq=112  ... dentro -> PRESENTE  acum  29.2s
```

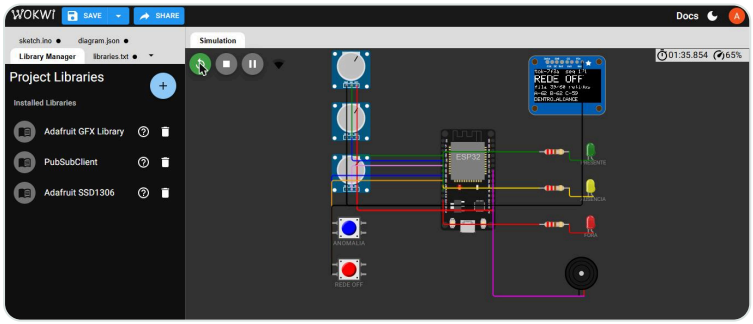
Defeito encontrado neste teste

Quando o 101 chega antes do 100, o servidor vê uma lacuna e conta "1 perdido". O 100 chega em seguida, mas a contagem não era desfeita: o relatório final dizia `6 perdidos`, quando só 5 se perderam. Corrigido depois da execução. O servidor agora guarda as lacunas e, quando um atrasado preenche uma, desconta e escreve `preenche a lacuna: nao estava perdido, so atrasado`. Há um teste para isso. O log acima é o da execução original, com o defeito.

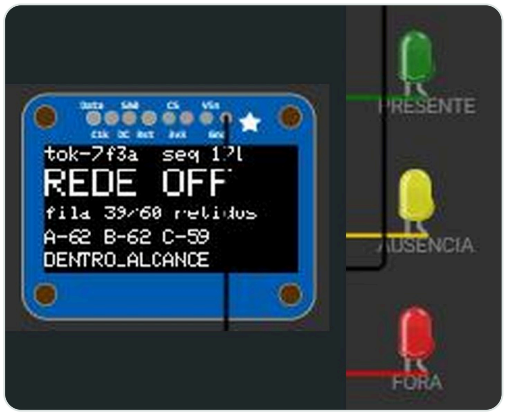
T4 Queda de rede: silêncio, retenção e reentrega

passou

Estado inicial	`PRESENTE`, t ≈ 68 s, acumulado 39,4 s.
Entradas	Botão vermelho uma vez; cerca de 25 s reais depois, de novo.
Valores e tempos	41 heartbeats retidos (seq 133 a 173, t de 68,8 a 88,8 s); a fila chegou a 39/60 na captura. Nenhum descarte por fila cheia.
Esperado	O aparelho continua medindo e mostra `REDE OFF` com a fila; LEDs de estado apagados. O servidor acusa silêncio e não decide nada. Na volta, recebe os retidos como atrasados, no tempo original, e o acumulado segue sem buraco.
Observado	`SILENCIO` 6 s depois do último heartbeat; `RETOMADO` e 41 linhas `ATRASADO … entra no passado e a sessao e recalculada`. O acumulado seguiu de 39,9 s a 59,9 s através do trecho retido. O estado continuou `PRESENTE`.
Por quê	Silêncio não é ausência. Um servidor ingênuo, vendo o aparelho sumir, poderia abrir uma ausência e cobrar orçamento. Este espera o dado: quando ele chega, mesmo atrasado, a sessão é refeita como se tivesse chegado a tempo.



Rede fora: o aparelho segue medindo e guardando. **Todos os LEDs apagados**, porque uma decisão velha não é exibida como atual.



OLED e LEDs ampliados.

aparelho · log-serial-aparelho.txt

```
### rede simulada: INDISPONIVEL
[REDE] indisponivel: heartbeat seq=133 retido (fila 1/60)
[REDE] indisponivel: heartbeat seq=134 retido (fila 2/60)
...
[REDE] indisponivel: heartbeat seq=173 retido (fila 41/60)
### rede simulada: RESTABELECIDA
[RETRY] 41 heartbeat(s) retidos entregues com atraso
```

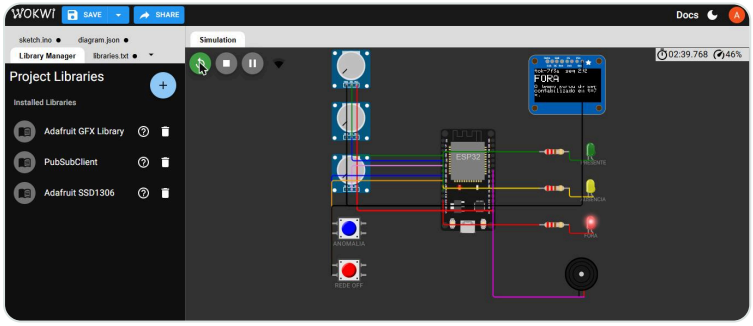
servidor · log-servidor.txt

```
01:35:47 t=  68.3s  seq=132  ... dentro -> PRESENTE  acum  39.4s
01:35:53 SILENCIO      nenhum heartbeat ha 6 s - rede, bateria ou app. Nada e decidido sem dado; o que c
01:36:25 RETOMADO      heartbeats voltaram a chegar
01:36:25 ATRASADO      seq=133 retido no aparelho, medido em 76335 ms - entra no passado e a sessao e
01:36:25 t=  68.8s  seq=133  ... dentro -> PRESENTE  acum  39.9s
...  (40 linhas ATRASADO)
01:36:28 t=  88.8s  seq=173  ... dentro -> PRESENTE  acum  59.9s
```

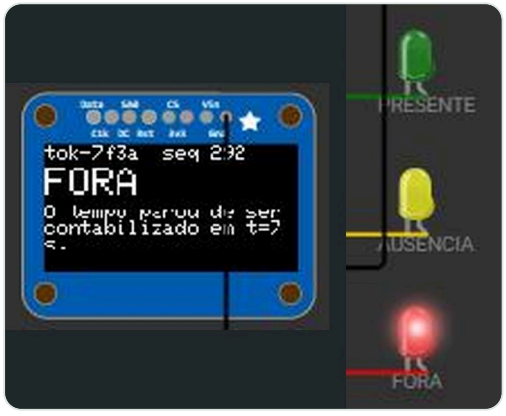
T5 Orçamento esgotado: aviso e estorno

passou

Estado inicial	Sessão nova `sb6a8`. PRESENTE desde t = 0, acumulado 6,6 s.
Entradas	Os três potenciômetros em 0: nenhuma âncora ouvida, fora do prédio. Mantidos assim por mais de 30 s.
Valores e tempos	RSSI `--` nas três; região `FORA_ALCANCE`. Ausência a partir de t = 6,6 s; avisos em 26,6 e 32,6 s; orçamento esgotado em t = 36,6 s.
Esperado	Os dois avisos e, ao fim do orçamento, `FORA` com LED vermelho, a mensagem de limite e a ausência inteira estornada: o acumulado volta a contar só a presença real.
Observado	`LIMITE_ESGOTADO` em t = 36,6 s: "O tempo parou de ser contabilizado em t=7 s." Estado `FORA`, LED vermelho, orçamento 0. Acumulado 6,6 s: nenhum segundo da ausência foi creditado.
Por quê	§6 do projeto técnico: sem retorno dentro do orçamento, o tempo desde o início real da ausência é estornado. A mensagem aponta esse início (t ≈ 6,6 s, arredondado no texto), e não o momento em que o limite estourou. É o horário que importa para o estudante contestar.



Limite esgotado: **FORA** e LED vermelho.



OLED e LEDs ampliados.

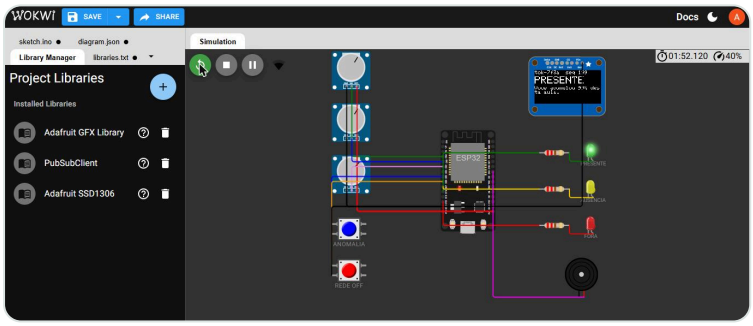
servidor · log-servidor.txt

```
01:37:31 t=  7.7s  seq=230  A -- B -- C -- FORA_ALCANCE  sem posicao fora  -> AUSENCIA_TOLERADA ac
   >>> PRESENTE -> AUSENCIA_TOLERADA em t=6.6 s (sinal fora da sala sustentado)
 AVISO AO ESTUDANTE  [LIMITE_ESGOTADO]  t=36.6 s
 "O tempo parou de ser contabilizado em t=7 s."
01:38:39 t=  37.2s  seq=288  A -- B -- C -- FORA_ALCANCE  sem posicao fora  -> FORA          acum  6.6s c
```

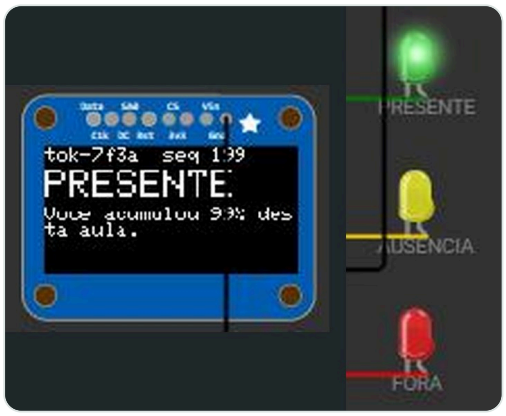
T6 Fim da aula: o resultado da sessão

passou

Estado inicial	Sessão `s31a2`, depois de T1 a T4.
Entradas	Nenhuma. A aula termina em t = 100 s.
Esperado	Aviso `FIM_DA_AULA` com o percentual; relatório final com o detalhamento; aprovação se ≥ 75 %.
Observado	"Voce acumulou 99% desta aula." Relatório: 71,1 s de presença + 27,5 s de ausência creditada = 98,6 s (98,6 %), **aprovado**. Orçamento usado 27,5 de 30 s.
Por quê	A presença perdida está toda no começo: a aula começa em t = 0 e a presença é confirmada em t = 1,4 s. Os outros 98,6 s estão cobertos por presença ou por ausência tolerada que se requalificou.



O aviso de fim de aula no aparelho.



OLED ampliado.

servidor · log-servidor.txt

```
------------------------------------------------------------------------
 RELATORIO FINAL DA SESSAO s31a2
 estado: PRESENTE   t=100.0 s de 100 s
 acumulado garantido: 98.6 s (98.6%)   provisorio: 0.0 s   -> APROVADO na aula (minimo 75%)
 presenca confirmada 71.1 s + ausencia creditada 27.5 s   orcamento usado 27.5 de 30 s
 contador ingenuo (ao vivo, sem dedup): 67.0 s
   t=   1.4s  FORA -> PRESENTE  (sinal na sala sustentado)
   t=   8.8s  PRESENTE -> AUSENCIA_TOLERADA  (sinal fora da sala sustentado)
   t=  36.3s  AUSENCIA_TOLERADA -> PRESENTE  (retornou dentro do orcamento)
 aceitos=188 duplicados=1 fora de ordem=1 saltos=2 (6 perdidos) atrasados=41 invalidos=0
------------------------------------------------------------------------
```

O "contador ingênuo" (0,5 s por heartbeat *dentro*, somado na chegada) daria 67,0 s, reprovando um estudante que só saiu com tolerância. Ele não enxerga a

ausência tolerada nem os intervalos irregulares entre heartbeats. O "6 perdidos" é o defeito do T3, já corrigido.

T7 Reinício do aparelho: silêncio e boot novo

passou

O que aconteceu	Não foi planejado: a simulação foi reiniciada logo depois de abrir a sessão `s31a2`, e a janela do Wokwi ficou oculta por alguns instantes (o navegador congela a simulação).
Observado	Dois heartbeats do boot anterior (seq 74, 75), `SILENCIO`, `RETOMADO` e o boot novo `a52e` começando em seq 1, **sem nenhuma duplicata**, encaixado em t = 1,0 s logo depois do último instante conhecido. O primeiro heartbeat do boot novo veio `INDETERMINADA` (janela da média ainda enchendo) e foi tratado como neutro.
Por quê	O `bootId` na identidade do evento e o encaixe da linha do tempo por boot.

servidor · log-servidor.txt

```
01:30:32 t=  0.5s  seq=75   ... DENTRO_ALCANCE (  6.0,  4.4) dentro -> FORA
01:30:38 SILENCIO      nenhum heartbeat ha 6 s - rede, bateria ou app. Nada e decidido sem dado; ...
01:30:51 RETOMADO      heartbeats voltaram a chegar
01:30:51 t=  1.0s  seq=1    ... INDETERMINADA  (  6.0,  4.4) neutro -> FORA
```

09 Testes automatizados

`python -m pytest servidor/testes -q` → **34 passed.** Não precisam de Wokwi nem de broker: o servidor é exercitado com um cliente MQTT falso, e cada resposta publicada é validada contra o próprio contrato.

GRUPO	O QUE GARANTE
Parâmetros	A escala comprime todos os tempos juntos (30 s → 1 s, 15 min → 30 s…)
Localização	A trilateração recupera o ponto em 4 posições; sinal forte na sala vizinha é *fora*; âncora não ouvida deixa neutro
Máquina de estados	Persistência na entrada; oscilação curta não derruba; neutro interrompe; entrada datada no início real
Orçamento	Ausência curta creditada só depois de requalificar; voltar por instantes não reabre crédito; avisos em 20 s e 26 s, uma vez só; esgotamento estorna tudo; orçamento por sessão; retorno a tempo não é punido pela demora da confirmação
Retroativo	Embaralhar o log não muda o resultado; lote atrasado corrige o passado; o acumulado nunca diminui (verificado a cada heartbeat de uma sessão inteira)
Fim da aula	Quem sai no fim soma o que assistiu
Servidor	Respostas válidas pelos schemas; estado retido e por aluno; campo desconhecido rejeitado; duplicata fora do log; fora de ordem na posição original; salto conta perdidos; atrasado que preenche lacuna não fica perdido; reboot não vira duplicata; lote atrasado recalcula; contador ingênuo infla com duplicatas

Um desses testes falhou na primeira execução e mostrou um problema real. Durante o segundo de confirmação de uma saída, o acumulado garantido subia meio segundo e depois voltava. A cauda ainda não confirmada passou a ser contada como provisória.

10 O que só apareceu rodando

ACHADO	CONSEQUÊNCIA	TRATAMENTO
Fora de ordem inflava os perdidos	Relatório com 6 perdidos quando eram 5	Lacunas por boot; o atrasado que preenche uma é descontado (T3)
O "fora de ordem" do protótipo de CO_2 nunca foi fora de ordem	Ele reenviava uma sequência *já processada*, e o gateway a classificava, corretamente, como duplicata. O teste adversarial daquela fronteira não exercitava o que dizia	A âncora de CO_2 agora segura o evento N e o entrega depois do N+1, como o aparelho. Compilado no Wokwi
Acumulado garantido não era monotônico por meio segundo	Subia e voltava durante a confirmação da saída	Cauda não confirmada vai para o provisório
Janela oculta congela a simulação	Relógio parado em 14,5 s; apertos de botão feitos nesse intervalo duram 0 ms simulados e não são vistos	Na apresentação, a janela do Wokwi fica em primeiro plano. O servidor mostrou `SILENCIO` nas quatro vezes em que isso aconteceu, sem decidir nada

11 O que foi validado e o que não foi

VALIDADO NO SIMULADOR	EXIGIRIA HARDWARE E AMBIENTE REAIS
Contrato, sequência, deduplicação, fora de ordem, salto, silêncio	Propagação de rádio real: multipercurso, corpo humano, parede, orientação do aparelho
Histerese de região, trilateração, polígono, persistência	Calibração dos limiares em dBm por levantamento na sala (cantos, porta, corredor, vizinha)
Orçamento, requalificação, estorno, avisos no momento certo	Cadência real: heartbeat de 60 a 90 s no Android e eventos de região no iOS
Retenção local e reentrega com recálculo retroativo	Fila sobrevivendo a reinício do aparelho (aqui é volátil)

Declaração obrigatória sobre a entrada

Os potenciômetros **não são rádio**. Não houve aquisição nem validação de RSSI real: o Wokwi não simula propagação, não há BLE, distância nem parede. São fontes de valor controláveis entre −105 e −35 dBm.

Fora do escopo deste marco: token rotativo da âncora, assinatura do heartbeat com a chave do aparelho, pareamento por QR, variância de IMU, motivo da ausência (`bateria` , `app_morto`) e `PAUSA_PROGRAMADA` , que depende da agenda.

12 Roteiro para a apresentação ao vivo

1 **Antes**

Rede com a porta 1883 liberada (na faculdade ela é bloqueada; use rede móvel). `python servidor.py` rodando. Wokwi aberto e **em primeiro plano**.

2 **Contexto, 1 min**

Seção 02: quem mede, quem decide, o que atravessa a fronteira.

3 **Play**

Em ~10 s o OLED mostra `PRESENTE` e o LED verde acende. Aponte o `eventId` no serial e a mesma linha no servidor.

4 **Sala vizinha**

Gire os potenciômetros para 481 / 652 / 531. Mostre que a região continua `DENTRO_ALCANCE` e o servidor diz *fora*: é o polígono.

5 **Avisos**

Espere o buzzer (20 s de ausência). Volte ao centro depois do segundo aviso. O LED verde volta.

6 **Adversarial**

Botão azul três vezes, esperando uns segundos entre elas. Mostre `DUPLICADO` , `FORA DE ORDEM` e `SALTO` no servidor.

7 **Rede**

Botão vermelho, espere o `SILENCIO` no servidor, aperte de novo e mostre as linhas `ATRASADO` .

8 **Alteração ao vivo**

Uma destas, cada uma de uma linha: mudar `escala` no config; mudar `LIMIAR_ENTRADA` no sketch; ou acrescentar `"bateria":63` ao heartbeat e mostrar o servidor rejeitando pelo

contrato.

CENÁRIO	A	B	C	RESULTADO
Centro da sala	628	628	667	(6,0; 4,4), dentro
Sala vizinha	481	652	531	(17,1; 4,4), fora com −60 dBm
Fora do prédio	0	0	0	nenhuma âncora ouvida

Teclas do servidor: `p` painel · `n` nova sessão (use antes de cada demonstração) · `q` sair.

13 Perguntas prováveis

P. Por que o ESP32 é o celular, e não a âncora?

Porque a âncora nunca escuta aparelhos (decisão fechada nº 3). Quem mede o sinal é o celular. A fronteira que carrega a permanência é, portanto, celular → servidor. A âncora aparece como o sinal que o celular ouve.

P. Se o servidor trilatera, para que a histerese no aparelho?

Para não mandar ao servidor a oscilação de quem está na borda do alcance, e porque é assim que o iOS funciona: o sistema decide a entrada e a saída da região e acorda o app. O polígono fica no servidor porque o aparelho não conhece a planta.

P. O que acontece se chegar um heartbeat de 20 s atrás?

Ele entra na posição dele no log e a sessão é refeita. Se ele mostra que o estudante estava na sala, o que parecia ausência vira presença. O acumulado garantido nunca cai; o que muda é o provisório.

P. Por que não somar 0,5 s a cada heartbeat?

Duplicatas contariam duas vezes, atrasados cairiam fora do lugar, e a regra de que a ausência começa no início real, e não na confirmação, seria impossível. Na sessão gravada, esse contador daria 67 s; o correto é 98,6 s.

P. E se o celular morrer?

O servidor vê silêncio e não decide nada sem dado. Distinguir bateria de app morto (o atributo `motivo`) está fora deste marco.

P. O RSSI de um potenciômetro vale alguma coisa?

Como grandeza física, não. Ele exercita a lógica com valores controláveis e repetíveis. Os limiares reais sairiam de um levantamento na sala.

P. Por que o `eventTimeMs` é a linha do tempo, se o relógio do aparelho não é confiável?

Porque aqui o problema é o oposto: o relógio do Wokwi anda a 10 %–80 % do real, e medir 30 s pelo relógio do servidor daria outra coisa a cada execução. O aparelho só contribui com diferenças dentro do mesmo boot, e o carimbo do servidor fica gravado em cada linha do log.

Parte II · segunda fronteira

Âncora de CO_2 ↔ gateway: a defesa contra o conluio em massa

Construída antes da fronteira principal e mantida como complemento. A turma deixa os celulares na sala e vai embora: todas as defesas de rádio confirmam a presença. O CO_2 da sala não. Pastas `co2-ancora/` e `co2-gateway/`.

C1 O problema que este protótipo resolve

O sistema **Presença** mede a permanência de estudantes em sala por rádio: três âncoras transmitem, o celular ouve, e a posição sai por trilateração. Contra quase toda fraude isso funciona — token rotativo impede forjar sinal, assinatura impede falsificar identidade.

Existe uma fraude, porém, que **nenhuma defesa de rádio alcança**: a turma inteira deixa os celulares na sala e vai embora. Os aparelhos estão genuinamente ali. A trilateração confirma, o token confere, a assinatura bate. O rádio não tem como saber que não há ninguém.

> **A ideia**
>
> Pessoas respiram e produzem CO_2. Se o rádio declara 30 estudantes e o CO_2 diz que a sala está vazia, alguém está mentindo — e **o ar não mente**.

É isso que esta âncora faz: cruza duas fontes independentes de evidência e acusa a contradição. Nenhuma das duas decide sozinha.

O ciclo completo exigido pela atividade

O protótipo implementa o caminho inteiro, e vale saber apontar cada etapa no código:

ETAPA	ONDE ACONTECE
Fenômeno	Concentração de CO_2 na sala (representada pelo potenciômetro)
Sensor / entrada	`analogRead(34)` — ADC de 12 bits
Validação	Faixa física + rejeição de salto implausível + média móvel
Evento estruturado	`sala.ambiente.v1` em JSON, no serial e no MQTT
Estado e regra	Histerese, persistência, validade e verificação cruzada
Decisão	● COERENTE · ● ATENÇÃO · ● INCOERENTE
Atuação	LEDs e display na âncora; alerta no gateway

C2 A fronteira entre os dois componentes

O Marco 2 pede uma integração real entre *dois* componentes — não o sistema inteiro. A fronteira escolhida é a que existe de verdade no projeto: a âncora da sala falando com o gateway do prédio.



Repare que a seta **vai e volta.** O gateway não só recebe: ele devolve a contagem de estudantes que o subsistema de rádio declara. Sem esse dado a âncora não teria como fazer a verificação cruzada, porque ela não tem acesso ao rádio.

Quem decide o quê, e por quê

DECISÃO	ONDE	MOTIVO
Estado da sala (coerente ou não)	Âncora	Depende de histórico de segundos que não vale a pena trafegar pela rede
O que fazer com esse estado	Gateway	Depende de contexto que a âncora não tem: turma, horário, histórico do semestre
Qual horário vale para o registro	Gateway	O `eventTimeMs` do ESP32 é tempo desde o boot, e relógio de dispositivo em campo não é confiável

C3 Todos os arquivos, e por que cada um existe

```
marco-02/
├── GUIA.md                ← este guia em Markdown
│
├── contrato/              ← O IDIOMA que os dois componentes falam
│   └── v1/                    "v1" é a versão — e ela faz parte do nome
│       ├── sala.ambiente.v1.schema.json
│       ├── gateway.contagem.v1.schema.json
│       └── exemplos/          quatro instâncias válidas
│
├── co2-ancora/              ← O QUE RODA NO ESP32
│   ├── sketch.ino             o programa, ~530 linhas de C++
│   ├── diagram.json           o circuito descrito em texto
│   └── libraries.txt          as três bibliotecas externas
│
├── co2-gateway/           ← O QUE RODA NO NOTEBOOK
│   ├── consumidor.py          o gateway em Python
│   ├── requirements.txt       paho-mqtt e jsonschema
│   └── config.exemplo.json    modelo de configuração
│
└── evidencias/            ← as imagens deste guia
```

Por que o contrato tem uma pasta só para ele

Esta é a pasta mais importante conceitualmente, e a que mais gente esquece de fazer.

Quando dois programas diferentes conversam, eles precisam concordar **antes** sobre o formato das mensagens. Se o ESP32 mandar `"valor"` e o Python esperar `"value"`, nada funciona — e o erro só aparece em execução, no pior momento possível.

O contrato resolve isso sendo um **arquivo**, não um acordo verbal. E não é decorativo: o consumidor **lê o schema do disco em tempo de execução** e valida cada mensagem contra ele.

ARQUIVO	O QUE É	QUEM USA
`sala.ambiente.v1.schema.json`	Descrição formal do evento: campos, tipos, faixas, obrigatoriedade	O consumidor valida cada mensagem contra ele
`gateway.contagem.v1.schema.json`	Mesma coisa para o comando de volta	Documentação; o ESP32 valida de forma simplificada, por limite de memória
`exemplos/*.json`	Uma instância válida por situação	Para entender e para testar sem ligar nada

Por que a versão está no nome

No dia em que o formato mudar de um jeito incompatível, cria-se `sala.ambiente.`**`v2`** e a `v1` continua existindo. Os dispositivos antigos seguem funcionando enquanto migram. Se a versão não estivesse no nome, você quebraria todo mundo de uma vez.

Os três arquivos do produtor são o projeto do Wokwi

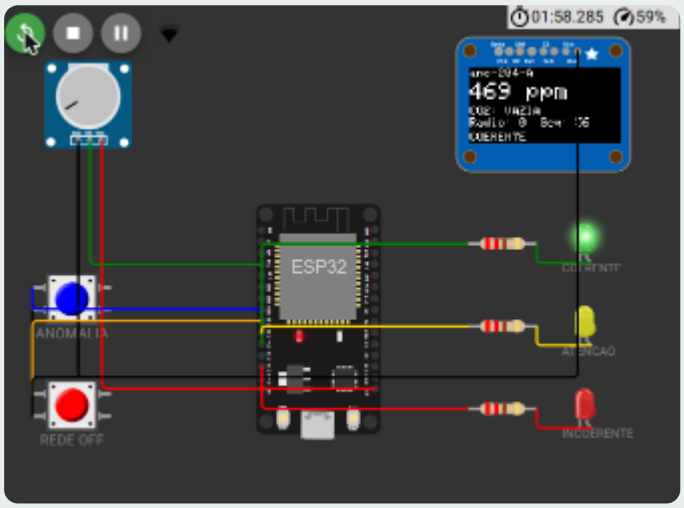
Se você abrir um Wokwi vazio e colar `sketch.ino`, `diagram.json` e `libraries.txt`, tem o protótipo inteiro de volta. São também três dos arquivos exigidos no ZIP da entrega individual.

O `diagram.json` merece atenção: ele é o **circuito descrito em texto**. Diz quais peças existem, onde ficam na tela e quais fios ligam o quê. O Wokwi desenha a partir dele — não existe um editor gráfico por trás guardando outra coisa.

Arquivos que nascem em execução

ARQUIVO	QUANDO APARECE	VERSIONADO?
`config.json`	Quando você copia o exemplo	Não — é seu, pode ter outro broker
`recebidos.log`	Toda execução do consumidor	Não — mas é a sua evidência de que funcionou

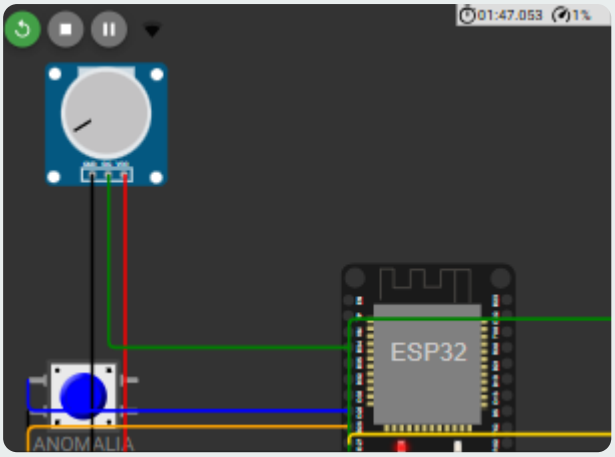
C4 O circuito, peça por peça



Visão geral

Onze componentes. Na foto, capturada com a simulação em execução, o display mostra `469 ppm` e o LED verde está aceso.

`GPIO34 potenciômetro` `GPIO25 botão anomalia` `GPIO26 botão rede` `GPIO14 LED verde` `GPIO27 LED amarelo` `GPIO13 LED vermelho` `GPIO21/22 display`



Potenciômetro — a entrada de CO_2

O que é: um divisor de tensão variável. Girando o botão, a tensão no pino `SIG` varia continuamente entre 0 V e 3,3 V.

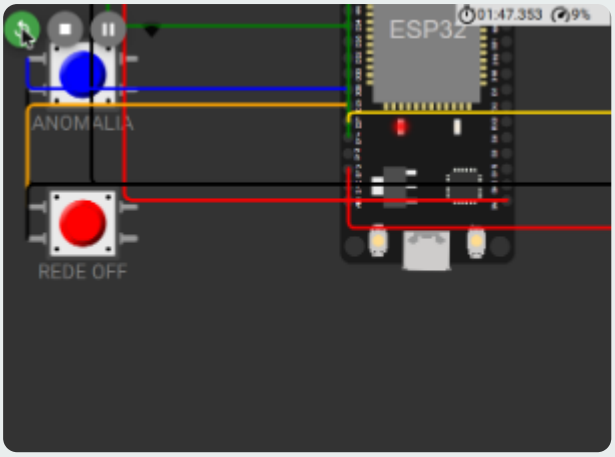
Como está ligado: `VCC` → `3V3`, `GND` → `GND`, `SIG` → `GPIO34`.

Como o código lê:

```
int cru = analogRead(PINO_POT);          // 0..4095 — ADC de 12 bits
return 380.0 + (cru / 4095.0) * (2200.0 - 380.0);
```

> **Declaração obrigatória**
>
> **Não é um sensor de CO_2.** Não há aquisição de grandeza física real, calibração nem resposta química. É uma fonte de valor controlável que eu declaro estar representando CO_2 entre 380 e 2200 ppm. O enunciado autoriza a substituição desde que ela seja declarada.

Por que o GPIO34: porque é entrada do **ADC1**. Os pinos do ADC2 param de funcionar quando o Wi-Fi está ligado — e este projeto vive de Wi-Fi. Usar um pino do ADC2 seria um bug silencioso: leituras zeradas assim que o rádio subisse.



Os dois botões — as condições de falha

Cada botão tem um lado no pino do ESP32 e o outro no `GND`. Não há resistor externo porque o código usa o **pull-up interno** do chip:

```
pinMode(PINO_BTN_ANOMALIA, INPUT_PULLUP);
```

Consequência que confunde na leitura do código: o pino lê `HIGH` quando o botão está *solto* e `LOW` quando está *pressionado*.

Botão azul — injeta anomalia de sequência

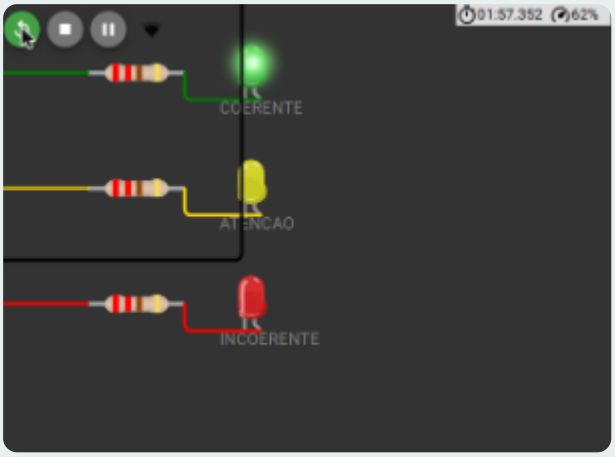
Cada aperto arma uma anomalia, aplicada na publicação seguinte, ciclando entre três. É o **teste adversarial obrigatório** da matrícula terminada em 8.

APERTO	ANOMALIA	O QUE A ÂNCORA FAZ
1º	`duplicata`	Publica de novo com a *mesma* sequência
2º	`fora-de-ordem`	Publica com sequência 3 números atrás
3º	`salto`	Pula 5 números na sequência

Botão vermelho — simula queda de rede

Com a rede fora, o ESP32 **continua amostrando e decidindo**, guarda os eventos numa fila de 20 posições e preserva o horário original de cada um. Ao apertar de novo, drena a fila marcando `bufferizado: true`.

Debounce: um botão físico não fecha o contato de forma limpa — nos primeiros milissegundos ele treme e gera dezenas de transições. Sem os 50 ms de debounce, um aperto viraria cinco anomalias armadas.



Os três LEDs e os resistores — a atuação

Cada LED segue o mesmo caminho: `GPIO → resistor 220 Ω → ânodo`, e o `cátodo → GND`.

LED	PINO	SIGNIFICA
Verde	`GPIO14`	CO_2 e contagem do rádio concordam
Amarelo	`GPIO27`	Dados insuficientes ou suspeita ainda não confirmada
Vermelho	`GPIO13`	O rádio declara ocupação que o CO_2 não sustenta

Por que 220 Ω: um LED é um diodo — ligado direto, conduz sem limite e queima, podendo levar junto o pino. O resistor limita a corrente a `(3,3 − 2,0) / 220 ≈ 6 mA`, confortavelmente abaixo dos 40 mA que o pino suporta.

Por que GPIO13 e não GPIO12: o GPIO12 é um **pino de strapping** (MTDI). Se estiver em nível alto no boot, o chip tenta reconfigurar a tensão da memória flash e pode não inicializar.



Display OLED — a observabilidade

Um SSD1306 de 128×64 pixels, monocromático, que comunica por **I²C** — um barramento de dois fios: `SDA` (dados) no `GPIO21` e `SCL` (relógio) no `GPIO22`. Responde no endereço `0x3C`.

Lendo a tela da foto, linha por linha:

`anc-204-A`	Identificação da âncora
`469 ppm`	CO_2 depois da média móvel
`CO2: VAZIA`	Ocupação inferida só pelo CO_2
`Radio: 0 · Seq: 51`	Contagem declarada pelo gateway e nº do evento
`COERENTE`	Estado final da verificação cruzada

> **Entenda esta tela**
>
> O estado é **COERENTE mesmo com a sala vazia** porque o rádio está declarando **zero** estudantes. Não há contradição entre "não tem ninguém" e "o rádio não vê ninguém". A incoerência só nasce quando o rádio afirma 10 ou mais *e* o CO_2 desmente.

Degradação: o código guarda se a inicialização do display deu certo e só desenha se deu. Display ausente ou com defeito não trava o resto — é decisão de projeto, não acaso.

O ESP32 — o cérebro

CARACTERÍSTICA	POR QUE IMPORTA AQUI
Dois núcleos a 240 MHz	Sobra folga para o filtro e o MQTT rodarem juntos
Wi-Fi integrado	É o que permite falar MQTT sem hardware adicional
ADC de 12 bits	Lê o potenciômetro com 4096 níveis
ADC2 inutilizável com Wi-Fi	Por isso o potenciômetro está no GPIO34
Pinos de strapping (0, 2, 12, 15)	Evitados para não atrapalhar o boot
`millis()` desde o boot	Não há relógio de tempo real: por isso `eventTimeMs` não é tempo absoluto

No Wokwi o ESP32 conecta na rede virtual `Wokwi-GUEST`, que tem saída real para a internet. É assim que ele alcança o broker MQTT público.

C5 Como o programa do ESP32 funciona

Amostragem e validação — a cada 400 ms

São **duas rejeições diferentes**, e vale saber distinguir:

```
// Rejeição 1 — leitura impossível: fora da faixa física do sensor
if (bruto < 350.0 || bruto > 5000.0) { amostrasInvalidas++; return false; }

// Rejeição 2 — ruído: gás não muda 300 ppm em 400 ms
if (temLeituraValida && fabs(bruto - ppmFiltrado) > 300.0) {
  amostrasInvalidas++; return false;
}
```

A primeira pega um valor que *nenhum sensor real produziria*. A segunda pega interferência elétrica: a concentração de um gás num ambiente fechado não muda centenas de ppm em quatro décimos de segundo — fisicamente não dá tempo.

A amostra aceita entra numa **média móvel de 5 posições**, que suaviza o que restou de ruído.

Camada 1 — ocupação pelo CO_2

Três mecanismos, todos dependentes de estado. Nenhum é comparação instantânea:

Histerese — dois limiares, não um

```
const float LIMIAR_OCUPADA = 800.0;   // sobe para OCUPADA acima disso
const float LIMIAR_VAZIA   = 600.0;   // desce para VAZIA abaixo disso
```

Com um limiar único de 700 ppm, um valor oscilando em torno dele faria o estado piscar dezenas de vezes por minuto. Com dois, existe uma **faixa morta** entre 600 e 800 onde o estado simplesmente não muda. É o mesmo princípio do termostato.

Persistência — a condição precisa se sustentar

Não basta cruzar o limiar: é preciso permanecer do outro lado por 6 a 8 segundos. O código acumula o tempo enquanto a condição vale e **zera assim que ela se quebra**.

Validade — o dado expira

Passados 10 segundos sem nenhuma leitura válida, a ocupação volta para `INDETERMINADA`. O sistema **esquece** o que sabia, em vez de continuar decidindo com base em dado velho.

Camada 2 — a verificação cruzada

```
bool condicaoSuspeita = (contagemRadio >= 10) && (ocupacao == OCUP_VAZIA);
```

Aqui estão as **duas entradas combinadas**: o CO_2, do sensor local, e a contagem do rádio, que chega pelo MQTT vinda do gateway. E a suspeita ainda precisa persistir por 10 segundos antes de virar acusação — antes disso o estado é ● **ATENÇÃO**, não ● **INCOERENTE**.

Rearme — evitando oscilação na atuação

Mesmo depois de tudo isso, o estado não pode mudar mais de uma vez a cada 3 segundos.

Montagem e publicação do evento

O JSON é montado com `snprintf`, sem biblioteca, para economizar memória — e vai para **dois lugares ao mesmo tempo**: o monitor serial (requisito da entrega individual) e o broker MQTT (a fronteira do Marco 2).

> **Armadilha que custa uma tarde**
>
> `mqtt.setBufferSize(512)` — o padrão do PubSubClient é **256 bytes**, e o nosso payload passa disso. Sem essa linha o `publish` falha *silenciosamente*: nenhum erro, nenhuma mensagem, simplesmente não chega.

Recebimento do comando — idempotência

```
if (cmdId.length() > 0 && cmdId == ultimoComandoId) return;   // já aplicado
```

O MQTT com QoS 1 garante entrega **ao menos uma vez** — a mesma mensagem pode chegar duas. Guardando o último `commandId` aplicado, uma reentrega não altera o estado. É o mesmo princípio que o consumidor usa com o `eventId`.

C6 Como o consumidor funciona

Validação contra o contrato, em tempo de execução

O consumidor **lê o arquivo de schema do disco** e valida cada mensagem contra ele. O contrato não é decorativo: se você acrescentar um campo ao evento no ESP32, o consumidor **rejeita** a mensagem, porque o schema declara `additionalProperties: false`.

> **A melhor alteração ao vivo**
>
> O professor pede uma pequena alteração durante a verificação. Acrescente um campo ao evento, mostre o consumidor rejeitando, e então versione para `v2`. Prova que o contrato está sendo validado de verdade, não apenas documentado.

As três detecções de anomalia

Esta é a parte que corresponde à responsabilidade individual declarada (**comunicação e resiliência**) e ao teste adversarial da matrícula.

DETECÇÃO	COMO FUNCIONA	O QUE O CONSUMIDOR FAZ
Duplicata	`eventId` já está no conjunto de vistos	Descarta sem alterar o estado
Fora de ordem	`sequence` menor que a maior já vista	Aceita e registra, mas não retrocede o estado
Salto	`sequence > maior + 1`	Conta quantos eventos faltaram e sinaliza

O conjunto de vistos é limitado a 500 entradas, descartando as mais antigas. Sem esse limite, um sistema rodando por semanas consumiria memória sem parar.

> **Detalhe que vale destacar**
>
> O consumidor **não usa** o campo `anomaliaInjetada` para detectar nada. Ele detecta pela sequência, exatamente como faria em produção. O campo existe só para deixar a demonstração legível.

O carimbo de tempo autoritativo

O `eventTimeMs` que vem do ESP32 é tempo *desde o boot*, não tempo absoluto — e o relógio de um dispositivo em campo não é confiável de qualquer forma. O tempo que vale para o registro é o do gateway, aplicado no instante da recepção.

O comando de volta

A cada 10 segundos o consumidor publica a contagem declarada pelo rádio. Durante a execução você digita um número no terminal para mudá-la — é assim que se provoca o estado ● INCOERENTE na demonstração.

C7 Um evento inteiro, do potenciômetro ao log

Vale decorar esta sequência — é a resposta para “explique como funciona”.

01 Você

Gira o potenciômetro para 470 ppm

02 ESP32

A cada 400 ms lê o ADC: `analogRead(34)` → 0..4095 → 470 ppm

03 ESP32 · validação

Está entre 350 e 5000? sim. Saltou mais de 300 ppm? não

04 ESP32 · filtro

Entra na média móvel de 5 amostras → `ppmFiltrado = 469`

05 ESP32 · camada 1

469 < 600 sustentado por 8 s → `ocupacao = VAZIA`

06 Gateway → ESP32

Havia publicado `contagem = 25`

07 ESP32 · camada 2

25 ≥ 10 **e** ocupação VAZIA → condição suspeita

08 ESP32 · persistência

A condição se mantém por 10 s → `estado = INCOERENTE`

09 ESP32 · atuação local

Apaga o LED verde, acende o vermelho, atualiza o display

10 ESP32 · evento

A cada 2 s monta o JSON com `eventId: "anc-204-A#63"`

11 ESP32 · saída

Escreve no monitor serial **e** publica no tópico MQTT

12 Broker

Entrega ao consumidor

13 Gateway · contrato

Valida contra o schema → ok

14 Gateway · dedup

Verifica o `eventId` no conjunto de vistos → novo

15 Gateway · sequência

Verifica lacuna e ordem → sem salto

16 Gateway · tempo

Carimba o horário de recepção — este é o que vale

17 Gateway · atuação remota

Detecta a mudança para INCOERENTE → **alerta na tela**

18 Gateway · registro

Grava a linha em `recebidos.log`

C8 Perguntas prováveis, com resposta pronta

P. Por que o potenciômetro e não um sensor de verdade?

Porque o Wokwi não simula química de gás, e o enunciado autoriza a substituição desde que declarada. O que estou validando é a **lógica de estado, o contrato e a resiliência** — não a aquisição do sinal. Isso está no cabeçalho do código e na seção de limitações do README.

P. O que acontece se chegarem dois eventos iguais?

O consumidor descarta o segundo, porque o `eventId` já está no conjunto de vistos, e o estado não muda. Posso demonstrar agora apertando o botão azul.

P. E se um evento chegar fora de ordem?

É aceito e registrado, mas não retrocede o estado. Retroceder seria pior que ignorar — significaria voltar para uma leitura que já sei estar desatualizada.

P. Por que a regra não é só `if (co2 < 600)`?

Porque um valor instantâneo oscila e geraria acusações falsas. A regra usa histerese de dois limiares, persistência de 6 a 10 segundos, média móvel de 5 amostras, expiração do dado após 10 segundos e um tempo mínimo de 3 segundos entre mudanças de estado.

P. Onde está o estado guardado?

Em três lugares, cada um com um motivo. No **ESP32**: a janela da média, os acumuladores de tempo e a fila de eventos não enviados. No **consumidor**: o conjunto de `eventId` vistos, a maior sequência e os contadores. No **contrato**: nada — contrato não guarda estado, ele descreve formato.

P. O que acontece se a rede cair no meio da aula?

O ESP32 continua amostrando e decidindo, e guarda os eventos numa fila de 20 posições com o horário original preservado. Quando volta, drena a fila marcando `bufferizado: true`, e o consumidor os trata como eventos *atrasados*, não como novos. Se a fila encher, descarta o mais antigo **e registra o descarte** — assim o intervalo aparece como incompleto, em vez de silenciosamente vazio.

P. Isso escala para um prédio inteiro?

O volume é de cerca de meio evento por segundo por sala. Sessenta salas dão 30 eventos por segundo, o que qualquer broker aguenta. O gargalo não é o dado, é o número de conexões Wi-Fi — e é por isso que, no projeto completo, as três âncoras de cada sala formam uma malha ESP-NOW e só a líder fala com o gateway.

Projeto **Presença** · Marco 2 e Atividade 03 · Software para Sistemas Ubíquos · Instituto de Informática, UFG · Prof. Dr. Otávio Calaça Xavier
Alany Gabrielly, matrícula 202105018 · capturas e logs da execução real em `marco-02/evidencias/`